Manuale Tecnico – TheKnife

1. Introduzione

Il presente Manuale Tecnico descrive in maniera **approfondita e motivata** l'architettura software, le scelte tecnologiche, le strutture dati e le soluzioni implementative adottate nello sviluppo dell'applicazione **TheKnife**, realizzata in linguaggio Java nell'ambito del Laboratorio Interdisciplinare A.

Il documento è destinato a un pubblico tecnico (docenti e assistenti del corso) e ha l'obiettivo di dimostrare non solo *cosa* è stato realizzato, ma soprattutto *perché* determinate scelte progettuali sono state adottate, evidenziandone la coerenza con i requisiti e con il contesto didattico del progetto.

---

2. Descrizione generale del sistema

TheKnife è un'applicazione **desktop stand-alone** che consente la ricerca, la consultazione e la gestione di ristoranti. Il sistema è ispirato a piattaforme reali di food discovery e recensione, ma è progettato con finalità didattiche, privilegiando chiarezza architetturale, semplicità e correttezza delle soluzioni implementative.

L'applicazione supporta tre tipologie di utenti:

- **Utente guest**, che può consultare i ristoranti senza autenticazione;
- **Cliente registrato**, che può inserire, modificare ed eliminare recensioni e gestire una lista di ristoranti preferiti;
- **Ristoratore**, che può inserire nuovi ristoranti, visualizzare il riepilogo delle valutazioni ricevute e rispondere alle recensioni.

Tutte le funzionalità sono eseguite localmente, senza comunicazioni di rete, database remoti o accessi concorrenti, in piena aderenza alle specifiche del laboratorio.

---

3. Obiettivi progettuali

Dal punto di vista tecnico e didattico, il progetto persegue i seguenti obiettivi:

- applicare i principi della **programmazione orientata agli oggetti**;
- progettare un sistema software strutturato e leggibile;
- separare in modo chiaro interfaccia grafica, logica applicativa e gestione dei dati;
- gestire la persistenza dei dati tramite file;
- sviluppare una GUI completa e utilizzabile;
- documentare il codice tramite JavaDoc.

Un obiettivo centrale del progetto è dimostrare la capacità di **motivare le scelte progettuali**, evitando soluzioni sovradimensionate o tecnologie non richieste dal contesto.

---

4. Architettura software

4.1 Significato di architettura monolitica

L'architettura adottata per TheKnife è di tipo **monolitico**. Con questo termine si intende un'architettura in cui tutte le componenti dell'applicazione (interfaccia grafica, logica applicativa e gestione dei dati) risiedono all'interno dello stesso processo e vengono eseguite come un'unica applicazione.

In un'architettura monolitica:

- non esistono servizi remoti o componenti distribuite;
- la comunicazione tra le parti avviene tramite chiamate dirette a metodi;
- il ciclo di vita dell'applicazione è unico.

4.2 Motivazione della scelta

La scelta di un'architettura monolitica è **intenzionale e motivata**:

- le specifiche del laboratorio **non richiedono** architetture client/server;
- non è necessario gestire accessi concorrenti ai dati;
- un'architettura monolitica riduce la complessità del progetto;
- è più adatta a un progetto didattico focalizzato su algoritmi, strutture dati e progettazione.

Questa scelta consente di concentrare l'attenzione sulla qualità del codice e sulla chiarezza della struttura, evitando complessità architetturali non richieste.

---

5. Separazione delle responsabilità

Pur essendo monolitica, l'applicazione è progettata secondo una **chiara separazione logica delle responsabilità**, ispirata ai principi del modello MVC:

- **Interfaccia grafica (GUI)**: gestione dell'interazione con l'utente;
- **Logica applicativa**: implementazione delle regole di funzionamento;
- **Gestione dei dati**: persistenza su file.

Questa separazione migliora:

- la leggibilità del codice;
- la manutenibilità;
- la possibilità di estendere il progetto in futuro.

6. Tecnologie utilizzate

6.1 Linguaggio di programmazione: Java

Il progetto è interamente sviluppato in **Java**, linguaggio orientato agli oggetti che offre:

- portabilità su diversi sistemi operativi;
- un ricco insieme di librerie standard;
- un modello a oggetti adatto alla progettazione modulare;
- ampia diffusione in ambito accademico e professionale.

Java rappresenta una scelta naturale e coerente con gli obiettivi del laboratorio.

6.2 Interfaccia grafica: Java Swing

L'interfaccia grafica dell'applicazione è realizzata utilizzando **Java Swing** (`javax.swing`). Swing è una libreria grafica inclusa nel JDK che consente lo sviluppo di applicazioni desktop multipiattaforma.

Motivazioni della scelta di Swing

La scelta di Swing è stata effettuata per i seguenti motivi:

- **assenza di dipendenze esterne**: Swing è incluso nel JDK;
- **integrazione nativa con Java**;
- **adeguatezza al contesto didattico** del progetto;
- semplicità di utilizzo per la creazione di finestre, pannelli e componenti;
- controllo diretto del flusso dell'applicazione.

Swing consente di realizzare un'interfaccia completa senza introdurre framework più complessi (come JavaFX), non necessari per gli obiettivi del progetto.

Le principali classi Swing utilizzate includono `JFrame`, `JPanel`, `JButton`, `JTextField`, `JTable` e `JOptionPane`.

---

7. Struttura dei package

Il progetto è organizzato nel package principale **theknife**, articolato in sottopackage:

- **theknife**: contiene la classe `TheKnife` con il metodo `main`;
- **theknife.gui**: classi Swing responsabili dell'interfaccia grafica;
- **theknife.model**: classi che rappresentano le entità del dominio;

- **theknife.data**: classi dedicate alla lettura e scrittura dei file;
- **theknife.util**: classi di supporto.

Questa organizzazione riflette la separazione delle responsabilità e facilita la comprensione del codice.

---

8. Modello dei dati

8.1 Utente

La classe `Utente` rappresenta un utilizzatore del sistema e contiene:

- username;
- password (memorizzata tramite hash);
- ruolo (cliente o ristoratore);
- eventuali dati aggiuntivi.

La gestione del ruolo è fondamentale per il controllo degli accessi.

8.2 Ristorante

La classe `Ristorante` modella un ristorante e include:

- nome;
- città e indirizzo;
- coordinate geografiche;
- fascia di prezzo;
- tipologia di cucina;
- servizi disponibili;
- lista delle recensioni.

8.3 Recensione

La classe `Recensione` rappresenta una valutazione lasciata da un cliente e contiene:

- autore;
- numero di stelle (1–5);
- testo;
- risposta del ristoratore.

---

9. Strutture dati

Per la gestione delle informazioni sono utilizzate strutture dati standard della libreria Java, in particolare `ArrayList`. Questa scelta è motivata da:

- semplicità di utilizzo;
- adeguatezza al numero di elementi gestiti;
- chiarezza del codice;
- buone prestazioni per le operazioni richieste.

---

10. Persistenza dei dati

La persistenza dei dati è realizzata tramite **file CSV**.

Motivazione della scelta

L'uso dei file CSV è coerente con le specifiche del laboratorio e consente di:

- evitare l'introduzione di database;
- concentrarsi sui meccanismi di I/O;
- mantenere il progetto semplice e facilmente distribuibile.

All'avvio i dati vengono caricati in memoria; ogni modifica comporta l'aggiornamento dei file.

---

11. Logica applicativa

La logica applicativa gestisce:

- ricerca dei ristoranti tramite filtri combinabili;
- gestione delle recensioni;
- calcolo della media delle valutazioni;
- controllo dei permessi in base al ruolo dell'utente.

Le funzionalità sono implementate tramite metodi chiari e direttamente collegati ai requisiti.

---

12. Sicurezza

Le password non vengono memorizzate in chiaro, ma tramite una funzione di **hash**, migliorando il livello di sicurezza dell'applicazione e rispettando le buone pratiche di sviluppo.

---

13. Gestione degli errori

L'applicazione utilizza blocchi `try-catch` per gestire input non validi e problemi di lettura/scrittura dei file, evitando arresti anomali e migliorando l'affidabilità del sistema.

---

14. Compilazione ed esecuzione

Il progetto non utilizza strumenti di build automatici come Maven. La compilazione e l'esecuzione avvengono tramite un IDE Java o tramite riga di comando, eseguendo la classe `TheKnife`.

---

15. Possibili miglioramenti futuri

Tra i possibili miglioramenti futuri:

- introduzione di test automatici;
- utilizzo di un database;
- miglioramento grafico dell'interfaccia;
- refactoring secondo i principi SOLID.

---

16. Conclusione

TheKnife rappresenta un progetto completo e coerente con gli obiettivi del Laboratorio Interdisciplinare A, dimostrando una corretta applicazione dei principi di progettazione software, una gestione consapevole delle tecnologie e una chiara motivazione delle scelte adottate.